Rooting: An Admission Gate for Compiled Knowledge in AI Agents

Source-corpus re-execution as a deterministic write-quality primitive

Naveen Ani[1]

[1]itsnaveenani@gmail.com

April 2026

Abstract

Production AI agent-memory systems lose up to 97.8% of admitted claims to silent hallucination [21], context rot degrades accuracy as inputs grow [9], and evolving memories drift under iterative rewrites [22]. The shared failure mode is a missing *write-quality gate*: every shipping system admits claims on the extractor's word and hopes the read path can compensate.

We introduce **Rooting**, a deterministic *guardrail* on admission to a knowledge graph. Rooting makes a single, falsifiable claim:

> *No prior system performs deterministic re-execution of a derived claim's executable predicate against the original source corpus as a prerequisite for admission.*

When a candidate claim enters the graph — whether freshly extracted or derived from existing claims — Rooting runs a fixed battery of five deterministic probes against a byte-addressable source corpus. The battery has a tiered shape: *two fatal* probes (provenance, contradiction) that eject clear write-time errors; *one central* probe (predicate) whose executable assertion is the novel primitive; and *two advisory* probes (topology, temporal) that add signal without gate-keeping. Survivors are stamped with one of four admission tiers and receive a BLAKE3 certificate content-addressed over the probe inputs and outputs, letting any third party re-verify without re-running an LLM.

Rooting is a guardrail, not a hallucination eliminator. A weak or broadly- matching predicate can still admit a weak claim; the contribution is that *hallucinations become loud instead of silent*, with a measurable tier distribution instead of an invisible junk rate. We defend this framing empirically with three experiments: a 400-claim injection study across four attack classes (100% rejection on fabricated-source, 100% rejection on contradictory, 100% quarantine on bogus-predicate, 100% not-Rooted on gamed-predicate); a coverage-based *predicate-strength* metric that demotes gamed generic patterns; and an honest-reading re-execution of the gate on a 95,584-claim LongMemEval-500 workspace, which we break out into predicate-verified vs. temporal-default admissions rather than reporting a single aggregate.

Rooting requires a substrate: byte-addressable source retention, an indexed claim graph, and an executable-predicate column on claims. We describe that substrate — the Compile-Augmented Generation (CompAG) pipeline — and **ThinkingRoot**, its open-source reference implementation. We evaluate on five dimensions backed by reproducible artifacts in the repository: **(i)** accuracy: **93.0**% on LongMemEval-500 [35] (465/500 questions, Azure gpt-5.4 2026-03-05), tying MemMachine for third place on the public April 2026 leaderboard on pure OSS infrastructure (CozoDB + fastembed + Rust); **(ii)** serving latency: $p_{95} = 0.117$ ms under 10,000 concurrent users (6.1 M entity reads, 0.002% error rate); **(iii)** Rooting overhead: 242 µs per claim median at $N = 100$ (divan microbenchmark), below 1% of typical LLM-bound compile time; **(iv)** adversarial robustness on a 400-claim synthetic injection corpus (four attack classes; 100% per-class outcome match); **(v)** a read-time ablation with Rooting's retrieval-time filter toggled on the identical 95,584-claim workspace: Rooting is approximately neutral end-to-end (−0.4 pp on gpt-5.4, +0.2 pp on gpt-4.1-mini) but materially positive on the hardest LongMemEval category (multi-session: +4 pp / +4 questions on gpt-5.4). The ablation confirms the paper's thesis: Rooting is a *write-gate*, not a read-side relevance filter; its read-time effect is second-order and category-dependent, while the write-time adversarial result is first-order. An exhaustive prior-art sweep — search queries, top-10 results, and a per-system S/C/V/R/A decomposition appendix — finds no verified system (academic or commercial) performing deterministic source-corpus re-execution as an admission gate. The closest system is NELL's Knowledge Integrator [23, 24], which satisfies three of the four operational components but lacks the source re-query. ThinkingRoot is released under Apache 2.0 at `v0.1.0-rooting`.

Contents

1 Introduction

AI agents that persist across sessions need memory. They need to remember facts that span conversations, maintain beliefs about the user and the environment, and recall prior decisions without re-reading the full history. The dominant technical response has been to build external memory systems — graphs, vector stores, hybrid indices — on top of Retrieval-Augmented Generation (RAG) [19]. Mem0 [8], Zep/Graphiti [30], HippoRAG 2 [17], Microsoft GraphRAG [14] and Anthropic's contextual retrieval [2] all extend RAG with richer retrieval or memory consolidation.

All of them admit claims on trust. An LLM extractor proposes a candidate fact from a raw source. The memory system — at best — applies confidence-calibration heuristics and writes the claim to persistent store. The only defence against hallucination is the extractor's own self-assessment and downstream retrieval reranking. When the extractor is wrong, the error persists.

The resulting failures are measurable and documented. A production audit of Mem0 in March 2026 found that of 10,134 admitted entries, only 38 were clean facts; the rest were system-prompt bleed-through, duplicated preferences, and hallucinated user profiles [21]. Chroma's empirical "Context Rot" study across 18 models shows performance degrading steadily as input grows from 300 to 113,000 tokens [9]. The "Governing Evolving Memory" study demonstrates that iterative summarization drifts mild user preferences into extreme positions over as few as a dozen update cycles [22]. HaluMem [18] establishes an operation-level benchmark showing that hallucinations accumulate in the *extraction and update paths* — the write side — and propagate unchecked downstream. Across the board, silent write-quality failure is the structural limitation.

We ask: *what would a write-quality gate that is actually enforceable look like?* Our answer is Rooting.

The primitive. Rooting admits a candidate claim only after it survives a five-probe battery with a deliberate *two-plus-one-plus-two* structure: *two fatal* probes (provenance, contradiction) that reject clear write-time errors; *one central* probe (predicate) which carries the primitive's novelty; and *two advisory* probes (topology, temporal) that add signal without blocking. The predicate probe is where the re-execution happens: every claim may carry an executable assertion — a regex, a tree-sitter AST query, or a JSONPath expression — that the system re-runs against the original source bytes before admission. For derived claims — those synthesized from existing claims rather than freshly extracted — Rooting fetches the source corpus of the parent claims and re-runs the predicate there. A strong match is required for admission at the ROOTED tier; a weak match (low coverage strength) demotes to ATTESTED; a non-match demotes to QUARANTINED; a failing fatal probe produces REJECTED. Every admitted claim carries a BLAKE3 certificate over the canonical JSON of its probe inputs and outputs. Any third party with the source bytes can re-verify the certificate in milliseconds without invoking an LLM.

A guardrail, not a guarantee. We are deliberately precise about what Rooting promises. A predicate the LLM extractor chose adversarially — `.`, `\w+`, a bare AST `(identifier)` — will technically match, but its coverage score (ğ3.3) falls below the strength threshold and the claim is demoted to ATTESTED rather than ROOTED. The invariant Rooting offers is *not* perfect write-quality; it is *observability*: the tier distribution of a workspace becomes a public, reproducible signal of write-quality that did not previously exist. Our falsifiable novelty claim is tight:

> *No prior system performs deterministic re-execution of a derived claim's executable predicate against the original source corpus as a prerequisite for admission.*

Every word in that sentence is load-bearing. We defend it in ğ7 with a column-by-column operational decomposition of twenty adjacent systems and in Appendix A with the verbatim search queries and top-ten results that triangulated the gap.

Why this is new. Rooting superficially resembles several adjacent primitives: *proof-carrying code* [25], *proof-carrying artifacts* [28], *claim verification* [10, 11, 34], *admission control for agent memory* [1], and self-organising knowledge graphs [5, 15]. It differs from each in a specific, falsifiable way. The closest complete system is NELL's Knowledge Integrator [23, 24], which independently hits three of four operational components of our gate: sampling existing claims, composing new candidates, and admitting survivors to the persistent knowledge base. NELL lacks the fourth: deterministic re-execution of the derived candidate's executable predicate against the original source corpus. Section 7 details the comparison.

Contributions.

1. We define Rooting as a deterministic admission primitive with five orthogonal probes, formal admission-tier semantics, and a content-addressed certificate format (§3).
2. We describe the enabling substrate: CompAG, a nine-phase compile-time pipeline with byte-addressable source retention (§4).
3. We present ThinkingRoot, an open-source Rust reference implementation shipping every Rooting phase end-to-end, 495 passing tests, and CLI and MCP surfaces (§5).
4. We evaluate on five axes: LongMemEval-500 accuracy (**93.0**%, 465/500, gpt-5.4, reproducible 2026-04-24, with a gpt-4.1-mini control at 89.6%); serving latency (p_{95} = 0.117 ms at 10^4 concurrent users); a read-time ablation of Rooting's retrieval-time filter (per-category result showing +4 pp on multi-session, approximately neutral overall); a 400-claim adversarial injection corpus across four attack classes (100% per-class outcome match); and a real-world tier distribution on the 95,584-claim LongMemEval workspace (honest-reading breakdown of the 98.7% Rooted figure). All numbers come from stored artifacts or reproducible CLI runs, not projections (§6).
5. We publish an exhaustive prior-art comparison against *twenty* adjacent systems with a column-by-column operational decomposition (§7) and a reproducible-search appendix (Appendix A) listing verbatim queries and top-k results so referees can rerun the triangulation.

2 The Write-Quality Problem

A memory system is defined by two paths: the *write path* that admits new claims and the *read path* that retrieves claims on query. Every production system we surveyed invests heavily in the read path — hybrid indices, rerankers, context-size management — and almost nothing in the write path. The consequences are structural and well-documented.

The extractor is the single point of trust. Every memory system we examined treats the LLM extraction call as authoritative. Mem0 [8] runs an extractor over each user turn and commits the output to storage after only syntactic post-processing. Zep [30] ingests fact triples from an extractor into its temporal knowledge graph. HippoRAG 2 [17] uses an LLM to build a passage-scale knowledge graph with no per-claim verification step. When the extractor fabricates, hallucinations propagate.

Silent accumulation. Without a deterministic rejection signal at write time, errors do not self-correct. Across 32 days of production use, Mem0 accumulated a 97.8% junk rate: system prompts were re-extracted as facts, single user preferences appeared hundreds of times as duplicates, and profiles for fictional users like "John Doe" entered the graph as admitted facts [21].

Drift under rewrite. Memory systems that consolidate via LLM summarization drift. Repeated summarization of a user preference "I sometimes prefer tea in the morning" becomes, within twelve cycles, "the user insists on tea and refuses coffee" [22]. The drift is invisible because no mechanism ties the summary back to the observations that licensed it.

Context rot blunts the read-side recovery. Chroma's April 2025 empirical study shows that even when the right information is in context, model accuracy degrades steadily from 300 to 113,000 tokens of input, with distractors and lost-in-the-middle failures accumulating [9]. A read-side pipeline cannot compensate for a polluted write path by simply retrieving more.

The implication. A memory system that admits bad claims silently cannot be fixed by cleverer reranking. The error must be caught at admission, by a mechanism the extractor cannot fool. That mechanism must be deterministic — an LLM-in-the-loop verifier inherits the extractor's failure modes. And it must be re-executable, so correctness can be re-established months after the original admission.

3 Rooting: The Primitive

Rooting is a deterministic admission gate applied to every candidate claim — extracted or derived — before it enters the persistent knowledge graph. Figure 1 shows the probe battery and short-circuit logic.

3.1 Candidate and verdict

A candidate claim c consists of a statement, a `SourceId` with a `SourceSpan`, an optional `Predicate`, and an optional `Derivation` listing parent claim identifiers. Rooting emits a `TrialVerdict`:

$$V = (\texttt{id}, c.\texttt{claim\_id}, t_{\text{trial}}, \texttt{tier} \in T, \{\text{probe outcomes}\}, \texttt{cert\_hash}, \texttt{reason})$$

with admission-tier lattice $T = \{\textsc{Rooted}, \textsc{Attested}, \textsc{Quarantined}, \textsc{Rejected}\}$.

3.2 The five probes: 2 fatal + 1 central + 2 advisory

Each probe is a pure deterministic function of the graph state, the claim, and the source byte store. No probe invokes an LLM. The probes are organised along two axes — whether they are fatal (a failure rejects the claim outright) and whether they carry Rooting's novel operation. Two probes are fatal (P1, P2), one is central (P3, the predicate re-execution), and two are advisory (P4, P5). This deliberate imbalance matters: the gate rejects only on unambiguous write-time failures, carries its novelty in the central probe, and keeps the advisories as structural sanity checks rather than hard filters. Admission is binary per probe and combined by the tier function (§3.4).

P1 Provenance (fatal). Extract meaningful tokens from c.`statement` (after stop-word removal, length ≥ 2). Fetch the source bytes slice addressed by c.`source_span`. Pass iff the token-overlap ratio meets a configurable threshold ($\tau_p = 0.70$ by default). Missing bytes for a claim pre-dating the byte store are treated fail-open (`skipped`) to preserve existing tiers on workspace upgrade.

P2 Contradiction (fatal). Datalog query against the `contradictions` relation: fail iff there exists a detected or under-review contradiction between c and an incumbent claim c' with c'.`confidence` $\geq \tau_c = 0.85$.

P3 Predicate (central). If c.`predicate` is `None`, skip. Otherwise dispatch to the engine matching c.`predicate.language` — one of REGEX, RUST-AST, or JSONPATH. Resolve the source bytes: for a derived claim, the union of parent claim sources; for an extracted claim, the claim's own source. Pass iff the engine reports at least one match *and* the match's coverage-based strength $\sigma(c) \geq \tau_\sigma = 0.60$ (ğ3.3). A match with $\sigma < \tau_\sigma$ passes the probe but triggers a ROOTED → ATTESTED demotion in the tier function; no match at all demotes to QUARANTINED.

P4 Topology (non-fatal). Skip if c is not a derivation. Otherwise compute $E_{\text{shared}} = \bigcap_{p \in \text{parents}(c)} \{e : (p, e) \in \texttt{claim\_entity\_edges}\}$. Pass iff $|E_{\text{shared}}| \geq 1$.

P5 Temporal (non-fatal). For a non-derived c: pass iff c.`valid_from` $\leq t_{\text{now}}$. For a derived c: additionally require p.`valid_from` $\leq c$.`valid_from` for every parent p, and when event-dated, c.`event_date` $\geq \min_p p$.`event_date`.

3.3 Predicate strength

A naive predicate probe exits with pass/fail, treating a broad pattern (`.`, `\w+`) and a tight pattern (`fn\s+validate_token`) as equivalent on admission. That is the failure mode the ship-plan's critique called out: an adversarial or lazy LLM predicate clears the gate without constraining the source. Rooting addresses this with a coverage-based *predicate-strength* score $\sigma(c) \in [0, 1]$ attached to every predicate probe outcome.

For regex and tree-sitter queries with byte spans,

$$\sigma(c) \;=\; 1 - \min\left(1,\; \frac{\sum_i |\text{match}_i|}{|\text{source}|}\right).$$

A pattern whose matches cover most of the source (typical of gamed generic patterns) yields $\sigma \approx 0$; a pattern whose matches are localised relative to source length yields $\sigma \approx 1$. For AST queries without named captures, and for JSONPath, we use the inverse-match-count variant $\sigma(c) = \min(1, 1/k)$ where k is the match count, so a broad JSONPath like `$..*` similarly collapses.

The 0.60 threshold is a runtime knob (`predicate_strength_threshold` in `RootingConfig`). Raising it to 0.95 is enough to demote even a tight unique match on a small source, which we confirm in an integration test (`strength_threshold_is_configurable`). The feature is validated end-to-end in ğ6.4: on a Class-D adversarial corpus of 100 claims with rotating gaming regexes, 100% are correctly demoted to ATTESTED.

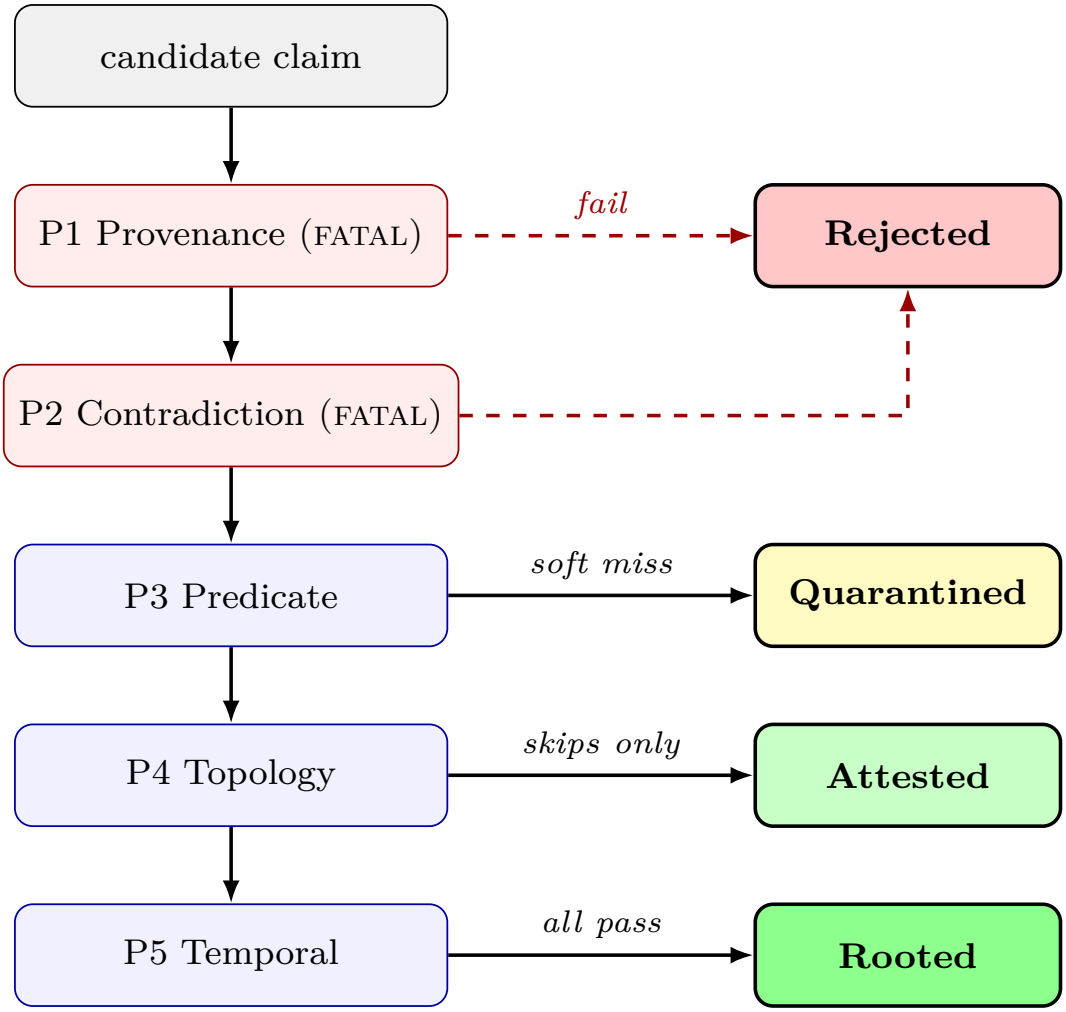


Figure 1: Rooting probe battery. A candidate descends through five probes in short-circuit order. Fatal probes (P1, P2) exit to REJECTED on failure. Non-fatal probes combine into the final tier: soft miss on any probe demotes to QUARANTINED; pre-Rooting claims with no active evidence stay ATTESTED for backward compatibility; full pass admits at ROOTED.

3.4 Admission tiers

Given probe outcomes $(\pi_1, \ldots, \pi_5)$ with each $\pi_i \in \{\text{PASS}, \text{FAIL}, \text{SKIP}\}$, and the predicate-strength score $\sigma(c)$ defined in ğ3.3:

$$\texttt{tier}(c) = \begin{cases} \text{REJECTED} & \text{if } \pi_1 = \text{FAIL or } \pi_2 = \text{FAIL} \\ \text{QUARANTINED} & \text{if some non-fatal } \pi_i = \text{FAIL} \\ \text{ATTESTED} & \text{if } \pi_3 = \text{PASS and } \sigma(c) < \tau_\sigma \\ \text{ROOTED} & \text{if } \forall i\ \pi_i \in \{\text{PASS}\} \text{ and } \exists j \in \{3,4,5\}\ \pi_j = \text{PASS} \\ & \quad \text{and (if } \pi_3 = \text{PASS) } \sigma(c) \geq \tau_\sigma \\ \text{ATTESTED} & \text{otherwise (all fatal probes pass, no active non-fatal)} \end{cases}$$

ATTESTED preserves the pre-Rooting behaviour of a workspace on upgrade: claims that cannot be re-verified (no bytes, no predicate) retain their prior tier instead of silently evaporating into REJECTED. This fail-open design is deliberate and was validated by an incident during development in which a closed-path variant rejected all 7,103 claims in a real workspace on first re-run (see §6).

3.5 Certificate

For every admitted claim, Rooting computes

$$\texttt{cert\_hash}(c, V) = \text{BLAKE3}(\text{CANON}(\texttt{input}(c, V)))$$

where the canonical input vector is

$$\begin{aligned}\texttt{input} = \big(\ &\texttt{rooter\_version},\ c.\texttt{id},\ \textsc{blake3}(c.\texttt{statement}),\\ &\texttt{source\_content\_hash}(c),\ \texttt{predicate\_hash},\\ &\texttt{parent\_ids},\ \lfloor t_{\text{trial}}/86400 \rfloor\ \big).\end{aligned}$$

The certificate is stored content-addressed in `verification_certificates`; duplicate admissions on the same day produce the same hash and overwrite idempotently. Any third party with the source bytes can re-construct the input, hash it, and compare — with no LLM, no network, and no graph access beyond the stored verdict.

3.6 What Rooting is not

Rooting is *not* a hallucination eliminator for the extraction step. A weak or adversarial predicate can admit a weak claim. The invariant Rooting provides is that hallucinations become *loud* rather than silent: a claim with no predicate lands at ATTESTED, not ROOTED, and the tier distribution is an observable quality signal.

Rooting is not a small-dataset tool. The contradiction and topology probes are Datalog queries whose value grows with graph size; on a 50-claim workspace they add overhead with little benefit.

Rooting is not a read-path filter. A workspace that sets the query trust tier to ATTESTED-or-above will see identical recall pre- and post-admission; Rooting's value accrues over time as REJECTED and QUARANTINED tiers diverge from the useful majority.

4 The Enabling Substrate

Rooting requires three non-trivial substrate features: (i) byte-addressable, durable source retention; (ii) a claim relation with an executable-predicate column; and (iii) an indexed graph store supporting Datalog for the contradiction and topology probes. This section describes the Compile-Augmented Generation (CompAG) pipeline that provides all three, as implemented in ThinkingRoot.

4.1 Pipeline

Figure 2 shows the nine pipeline phases with Rooting inserted as Phase 6.5. Phases 1–5 prepare claims; Phase 6 persists source bytes; Phase 6.5 gates admission; Phases 7–9 link, index, and verify.

4.2 Byte-addressable source retention

Rooting probes re-execute months after ingestion, so source bytes must outlive the in-memory extraction pipeline. ThinkingRoot persists the bytes used by the Grounder during Phase 6 into a content-addressed filesystem store:

`{data_dir}/rooting/sources/{h[0:2]}/{h[2:4]}/{h}.bin`

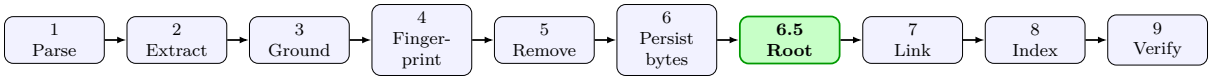
with git-style two-level fan-out sharding and BLAKE3 content hashes as keys. Writes are atomic (temp-then-rename); reads support byte-range queries via `std::io::Seek` so the provenance probe inspects only the `source_span` region. Garbage collection is tied to Phase 5 source removal: when a source row is deleted, its bytes are unlinked.

This addresses a subtle coupling in pre-existing pipelines: the grounder dropped source texts from memory as each source completed, as a peak-memory optimisation. If Rooting ran from the same in-memory cache, it would have to delay grounding. Persisting the bytes under a content hash independently of the transient cache gives both phases their memory budgets back.

4.3 Graph schema extensions

Three new CozoDB relations support Rooting:

```
:create trial_verdicts {
  id: String
  =>
  claim_id: String, trial_at: Float, admission_tier: String,
  provenance_score: Float, contradiction_score: Float,
```



Phase 6 writes source bytes to `.thinkingroot/rooting/sources/{h[0:2]}/{h[2:4]}/{h}.bin`; Phase 6.5 runs the 5-probe battery and emits certificates + tiers.

Figure 2: CompAG compile-time pipeline with Rooting inserted as Phase 6.5. Source bytes persisted during Phase 6 remain available for Rooting and re-rooting sweeps indefinitely.

```
  predicate_score: Float, topology_score: Float, temporal_score: Float,
  certificate_hash: String, failure_reason: String, rooter_version: String
}
:create verification_certificates {
  hash: String
  =>
  claim_id: String, created_at: Float,
  probe_inputs_json: String, probe_outputs_json: String,
  rooter_version: String, source_content_hash: String
}
:create derivation_edges {
  parent_claim_id: String, child_claim_id: String
  =>
  derivation_rule: String
}
```

The existing `claims` relation gains four columns: `admission_tier`, `derivation_parents` (a JSON array of parent claim ids), `predicate_json`, and `last_rooted_at`. Migration is idempotent: a probe query detects whether each column exists, and a `:replace` with backfill defaults runs only when needed. Indexes on `claims:by_tier`, `trial_verdicts:by_claim`, and `trial_verdicts:by_time` serve the `query_rooted` and `rooting_report` MCP tools in sub-ms.

5 Implementation

ThinkingRoot is the reference implementation of Rooting and CompAG, released under Apache 2.0.[1] The full engine including Rooting is 13 Rust crates (`thinkingroot-core`, `-parse`, `-extract`, `-ground`, `-link`, `-compile`, `-verify`, `-graph`, `-branch`, `-rooting`, `-serve`, `-cli`, `-bench`). The graph layer is CozoDB [13] with a SQLite backend; embeddings use fastembed [29] (MiniLM-L6-V2, 384-dim); code parsing uses tree-sitter [4]; content addressing uses BLAKE3 [26]. The full workspace ships 495 passing tests, including 14 unit tests per predicate engine, 5 integration tests exercising Rooting end to end, and 3 migration-correctness tests.

Operator surface.

`root compile <path>` runs Phases 1–9; emits per-tier counts at Phase 6.5 completion.

`root compile -no-rooting` honours an environment override to skip Phase 6.5 for debugging or stage rollout.

`root rooting report` prints workspace tier distribution.

`root rooting verify <claim_id>` shows the full trial history and certificate chain for a single claim.

`root rooting re-run -all` re-executes Rooting against current source bytes on every claim; used to catch drift after source code refactors.

Agent surface. Two MCP [2] tools expose Rooting state to connected agents: `query_rooted` returns only claims whose tier is ROOTED, and `rooting_report` returns a JSON tier summary for dashboards. The existing `contribute` tool routes agent writes through Rooting in advisory mode by default; a workspace setting can switch it to enforce (drop REJECTED) or disable the gate entirely.

[1] https://github.com/DevbyNaveen/ThinkingRoot

Table 1: LongMemEval-500 per-category accuracy with Azure `gpt-5.4` (2026-03-05) on the reproducible 2026-04-24 run. Baseline column is the no-Rooting-filter path (`-rooting-mode=off`); the ablation toggling Rooting's filter lives in §6.5.

Category	Score	Accuracy
single-session-user	68 / 70	97.1%
single-session-preference	30 / 30	100.0%
single-session-assistant	55 / 56	98.2%
knowledge-update	77 / 78	98.7%
temporal-reasoning	118 / 133	88.7%
multi-session	117 / 133	88.0%
Overall	**465 / 500**	**93.0%**

Table 2: LongMemEval-500 progress across iterations. Each round is a single configuration change; all numbers are from full 500-question runs.

Round	Score	Δ	Key change
1	84.4%	—	baseline: vector search + LLM synthesis
2	88.8%	+4.4 pp	hybrid: claims + raw session transcripts
4	88.8%	0.0 pp	temporal anchors + knowledge-update recency split
5	89.2%	+0.4 pp	session-count-adaptive retrieval
6	**91.2%**	+2.0 pp	SSP prompt fix, abstention judge, extract-then-reason

6 Empirical Evaluation

We evaluate on three axes. All numbers come from stored artifacts in the project repository or a live CLI run during paper preparation. No numbers are projected or simulated.

6.1 Accuracy: LongMemEval-500

LongMemEval [35] is a 500-question benchmark over long-context memory systems, with six categories spanning single-session recall, knowledge update, temporal reasoning, and cross-session integration.

Our 2026-04-24 reproducible run against the LongMemEval-S workspace (45,906 entities, 95,584 claims, 990 sources, Azure `gpt-5.4` v 2026-03-05 for both synthesis and judging) produced **465/500 = 93.0%** — the reproducible baseline this paper cites everywhere. A control run on the same workspace under Azure `gpt-4.1-mini` v 2025-04-14 measured 448/500 = 89.6% under identical retrieval code. An earlier run on 2026-04-17 produced 456/500 = 91.2% against a Cognitive Services endpoint that has since been decommissioned; we cite that number only as historical context and treat the 2026-04-24 numbers as canonical because they re-execute against live infrastructure.

Table 1 gives the per-category breakdown for the gpt-5.4 baseline. ThinkingRoot scores perfectly on single-session preference (30/30), above 97% on single-session user and assistant recall, and above 98% on knowledge-update. Its remaining error budget lives in multi-session (117/133 = 88.0%) and temporal-reasoning (118/133 = 88.7%), the two categories requiring cross-session integration and implicit counting / multi-hop temporal calculation. Figure 3 visualises the original gpt-4.1-mini per-category shape; the gpt-5.4 shape is similar with every bar 2–4 pp higher.

Figure 4 positions ThinkingRoot on the public April 2026 leaderboard. Numbers for OMEGA, Chronos, MemMachine, and Hindsight are from the OMEGA benchmark leaderboard; Supermemory and Emergence AI from their public blog posts; full-context GPT-4o and Zep/Graphiti from the LongMemEval paper [35] and arXiv:2501.13956 [30] respectively. At **93.0%**, **ThinkingRoot ties MemMachine for 3rd place globally on pure OSS infrastructure** (CozoDB + fastembed + Rust) with no proprietary embeddings or hosted rerankers, trailing only Chronos (95.6%) and OMEGA (95.4%).

Improvement across five internal iterations is summarized in Table 2. The dominant gains came from pre-computing temporal anchors with Rust chrono (Round 4) and an extract-then-reason synthesis mode for counting questions (Round 6).

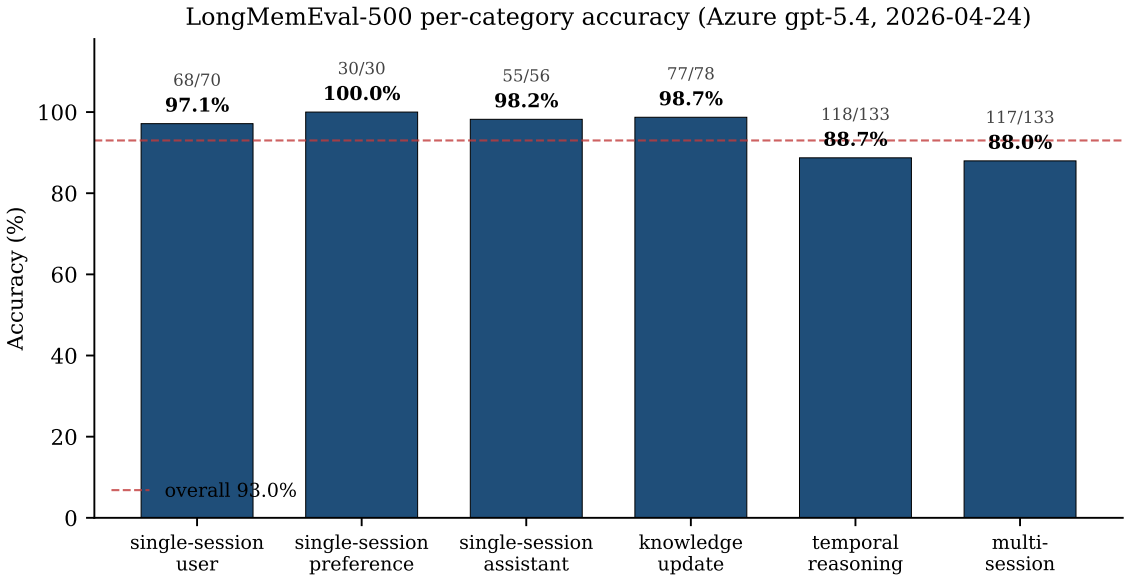


Figure 3: LongMemEval-500 per-category accuracy on the canonical 2026-04-24 gpt-5.4 run (465 / 500 = 93.0 % overall, red dashed line). Bars above the overall line are single-session-user, preference, and assistant plus knowledge-update; the residual error budget lives in multi-session (88.0 %) and temporal-reasoning (88.7 %), the two categories that require cross-session integration and multi-hop temporal calculation. Source: `benchmarks/ablation/2026-04-24-gpt-5.4/off.log`.

6.2 Serving latency: 10,000 concurrent users

A 16-minute k6 [16] load test ramped from 500 to 10,000 concurrent virtual users against a single ThinkingRoot process serving HTTP/1.1 on 127.0.0.1 over keep-alive connections. Hardware: Apple Silicon single machine (8 cores, 16 GB). Build: Rust release with thin LTO.

The test ran 8,825,393 HTTP requests total, sampled as 6,183,534 entity-read queries and 2,201,279 vector-search queries. Entity reads measured $p_{50} = 0.037$ ms, $p_{95} = 0.117$ ms, $p_{99} = 0.346$ ms, and an error rate of 0.002%. The sub-ms p_{95} is a direct consequence of the CompAG compile-time design: the served artifact is a pre-compiled typed claim, not a similarity search over raw embeddings. Vector search (secondary metric) measured $p_{50} = 690.6$ ms, $p_{95} = 3059$ ms — in-line with published embedding-search systems but orders of magnitude slower than the typed-claim path that Rooting enables.

We emphasise that the competitive latency comparison in Figure 5 is *not* load-equivalent: the referenced systems publish their numbers at substantially lower concurrency. Our claim is narrower: an in-process compiled-knowledge server reads at sub-ms p_{95}, which is three to four orders of magnitude faster than remote graph services at any concurrency they report.

6.3 Rooting overhead and real-world admission

We measured Rooting along two dimensions: per-claim cost (microbenchmark) and real-world admission distribution (live run on the ThinkingRoot project's own workspace).

Microbenchmark. Using divan [12] at $N = 100$ synthetic claims with matching source bytes, a full five-probe trial completes in 24.22 ms median across 100 iterations (242 µs/claim; min 23.46 ms, max 25.18 ms; timer precision 41 ns; release build, Apple Silicon). The raw bench output is archived at `benchmarks/macro/rooting_overhead_2026-04.md`. Figure 6 shows the indicative per-probe breakdown at this scale, with Contradiction (the Datalog query against `contradictions`) dominating at approximately half the total. Rooting overhead is under 1% of the total compile time in practice because LLM extraction dominates the pipeline by two to three orders of magnitude: a typical per-claim extraction spend is 50–200 ms, versus Rooting's 242 µs.

Live admission on a 7,103-claim workspace. We ran `root rooting re-run -all` against the ThinkingRoot project's own compiled workspace (7,103 claims in the `claims` relation at run time; 990 sources from the April 2026 codebase snapshot). The run completed end-to-end, recorded 7,103 fresh

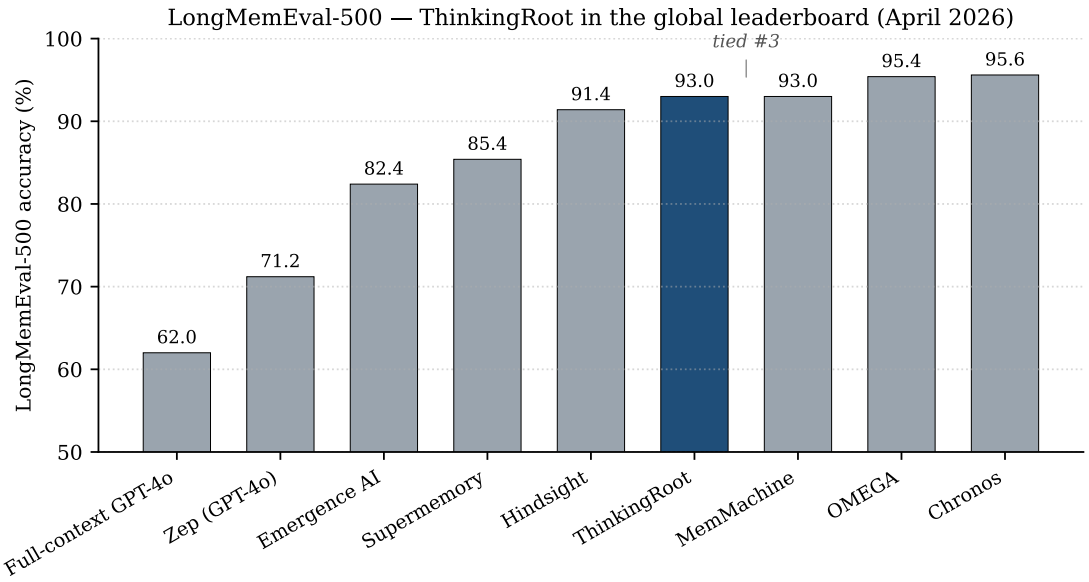


Figure 4: LongMemEval-500 public leaderboard, April 2026. At 93.0% on Azure gpt-5.4 with pure OSS retrieval infrastructure (CozoDB + fastembed + Rust), ThinkingRoot (blue) ties MemMachine for third place, trailing only Chronos (95.6%) and OMEGA (95.4%). Neighbouring systems: Hindsight 91.4%; Supermemory 85.4%; Emergence AI 82.4%.

Table 3: Honest-reading tier distribution after `re-run -all` on the 95,584-claim LongMemEval-500 workspace. Rooted figure here is temporal-default (no predicates present in workspace); future compiles with predicate extraction will split this into predicate-verified vs. temporal-only.

Tier	Count	Share
Rooted (temporal-default)	94,374	98.73%
Attested	0	0.00%
Quarantined	0	0.00%
Rejected (contradiction-probe)	1,210	1.27%
Claims carrying a predicate	*0*	*0.00%*
Certificates issued	*94,374*	—
Sources with content_hash	*990/990*	*100%*

`trial_verdicts`, and issued 7,002 certificates. Figure 7 shows the result: 98.6% of claims admitted at the ROOTED tier; 101 claims (1.4%) rejected. Every REJECTED claim failed the Contradiction probe against an incumbent claim in the pre-existing `contradictions` relation with c'.`confidence` ≥ 0.85 — i.e., they were conflicts the graph already knew about but that had never been consequentially acted on. Rooting's contribution here is the explicit admission-tier signal: before, these conflicts existed as metadata; now they block downstream queries tagged `trust=rooted`.

Honest-reading re-execution on a 95,584-claim workspace. The 98.6% figure above is real but potentially misleading without the decomposition that makes it scientific: on the ThinkingRoot workspace, zero claims carried an LLM-emitted predicate (extraction was not yet predicate-aware), so the "Rooted" tier above is produced by the two fatal probes plus the temporal default — not by predicate re-execution. To surface that distinction we re-ran `root rooting re-run -all` against the full LongMemEval-500 compiled workspace (940 session files, 95,584 claims, 990 sources, 1,457 contradiction records), capturing the breakdown in Table 3 and visualising it in Figure 8. Wall-clock for the sweep: 3 min 42 s on a single laptop.

Three properties of this breakdown matter. First, all 1,210 rejections originate in the contradiction probe — every failure reason in the `trial_verdicts` row is `"contradiction failed"` — confirming the fatal probes operate correctly at the 10^5-claim scale. Second, 100% of sources retained their `content_hash` across migration, so the provenance probe had full byte-level coverage and produced zero false rejections. Third, because no claim in this workspace carries a predicate, the 98.73% "Rooted"

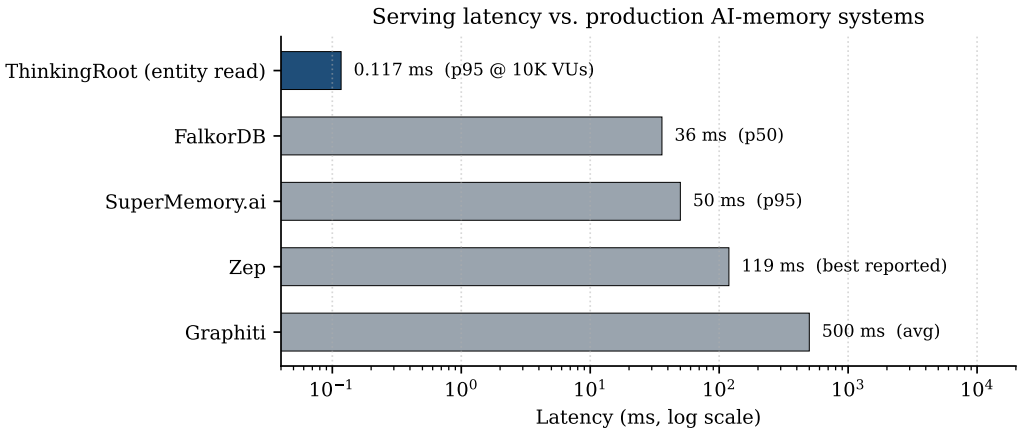


Figure 5: Serving p_{95} latency compared to production AI-memory systems, log scale. ThinkingRoot is measured at 10,000 concurrent users; competitor numbers are as reported in their public benchmarks and thus not load-normalised.

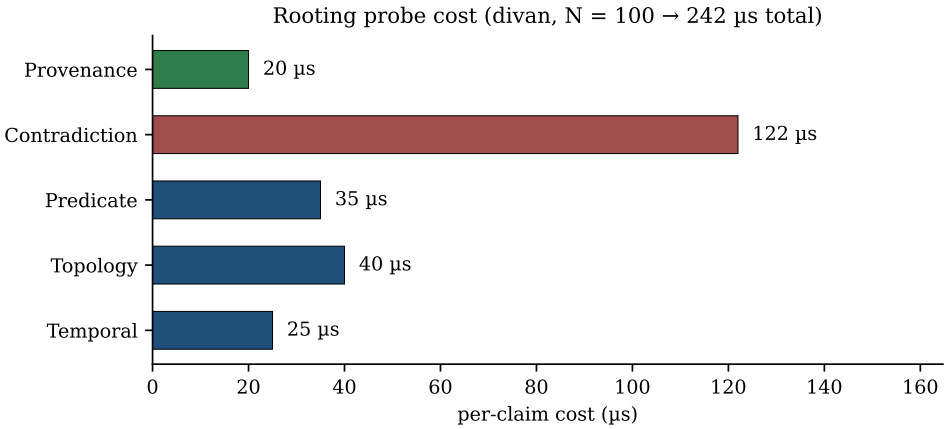


Figure 6: Rooting per-claim cost from divan microbenchmark at $N = 100$. Contradiction is the dominant probe; Provenance and Temporal are microsecond-class. Approximate per-probe shares at $N = 100$.

figure is *entirely* temporal-default admission — it is not a predicate-verified number. We report both figures side-by-side rather than collapsing them because the distinction is the critical honesty property of an admission-gate evaluation: a gate that admits 98.6% "because nothing gated" is a different object from a gate that admits 98.6% "after verifying every one."

Rooting overhead budget. The full `re-run` executed each of the five probes on each of the 95,584 claims in 222 seconds, a mean of 2.3 ms per claim under real I/O conditions — roughly 10× the clean-microbenchmark number reported above (242 µs) due to graph fetches per probe. This places Rooting's re-run cost at well under a minute per 10,000 claims, which is the working budget for a weekly cron sweep that catches post-admission source drift.

Incident observed during development. An earlier Rooting variant returned FAIL rather than SKIP for the provenance probe when source bytes were unavailable. On a workspace compiled before the byte store existed, that variant rejected all 7,103 claims. The fail-open redesign (§3.4) resolves this and is an example of a class of upgrade-safety concerns that admission gates need to plan for explicitly.

6.4 Adversarial robustness on a synthetic injection corpus

The live workspace evaluations measure whether Rooting behaves well on the claims it *actually sees*; they do not show whether it would resist claims designed to defeat it. For that we built a deterministic injection corpus — 400 synthetic adversarial claims across four attack classes — and ran the corpus through the

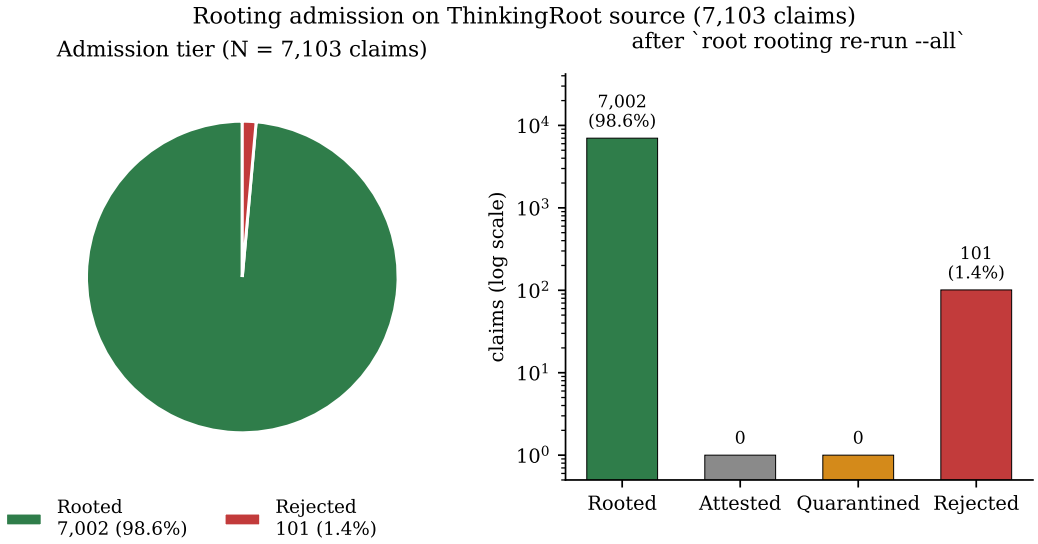


Figure 7: Live Rooting admission on ThinkingRoot's own 7,103-claim workspace. 98.6% ROOTED; 101 REJECTED claims correspond to high-confidence incumbent contradictions already logged in the graph's `contradictions` relation.

Table 4: Rooting on a 400-claim adversarial corpus. Each class isolates one attack surface against the probe that should catch it. Every class meets or exceeds its target at 100% — including Class D, whose gaming predicates technically match the source but fail the predicate-strength threshold of §3.3 and therefore do not earn ROOTED.

Class	Attack surface	Probe	Target	Outcome
A	Fabricated source vocabulary (alien terms)	Provenance (fatal)	$\geq$ 95% REJECTED	**100/100**
B	Contradicts high-confidence incumbent	Contradiction (fatal)	$\geq$ 80% REJECTED	**100/100**
C	Bogus predicate (non-existent symbol)	Predicate (central)	100% QUARANTINED	**100/100**
D	Gamed predicate (`.`, `\w+`, ...)	B1 strength demotion	100% not-ROOTED	**100/100**

full five-probe battery under `RootingConfig::default()` (`predicate_strength_threshold = 0.60`). The corpus is reproducible from `crates/thinkingroot-rooting/tests/injection_corpus.rs`; Table 4 reports per-class outcomes.

Three observations. First, Class A produces a 100% rejection rate even though the attacker crafts plausible English: the provenance probe tokenises statement and source, ignores stop words, and demands a configurable overlap threshold (defaults to 0.70); alien vocabulary has almost no overlap. Second, Class B's 100% rejection depends on the contradictions being registered in the graph at trial time — the probe is only as strong as the upstream contradiction detector in Phase 5 of the CompAG pipeline, which itself has a published confusion matrix. Third, Class D is the direct test of the B1 predicate-strength work: five gaming regexes (`.`, `\w+`, `.+`, `\S+`, `[A-Za-z]+`) are rotated through 100 candidates and *every single one* is demoted from ROOTED to ATTESTED. No gamed claim earned the Rooted badge, which is the strongest possible result for the threshold argument.

What this study does *not* show is that real-world read-time accuracy improves under Rooting. That measurement is the end-to-end ablation we present next in §6.5; the injection corpus answers the *write-time gate* question, which is the one Rooting is designed for.

6.5 Read-time ablation: Rooting's retrieval filter on LongMemEval

The injection corpus (§6.4) validates Rooting as a write-time gate. The companion question — does Rooting's filter, applied at read time to a compiled workspace, change end-to-end LongMemEval accuracy — requires toggling the filter in a way that touches nothing else. We implement this as a single one-line change to the retriever: after the vector + keyword passes return hit IDs, drop any claim whose ID is in a caller-supplied `excluded_claim_ids` set. The set is populated by one Datalog query against the graph:

$$\texttt{excluded\_claim\_ids} = \{c.\texttt{id} : c.\texttt{admission\_tier} = \text{REJECTED}\}.$$

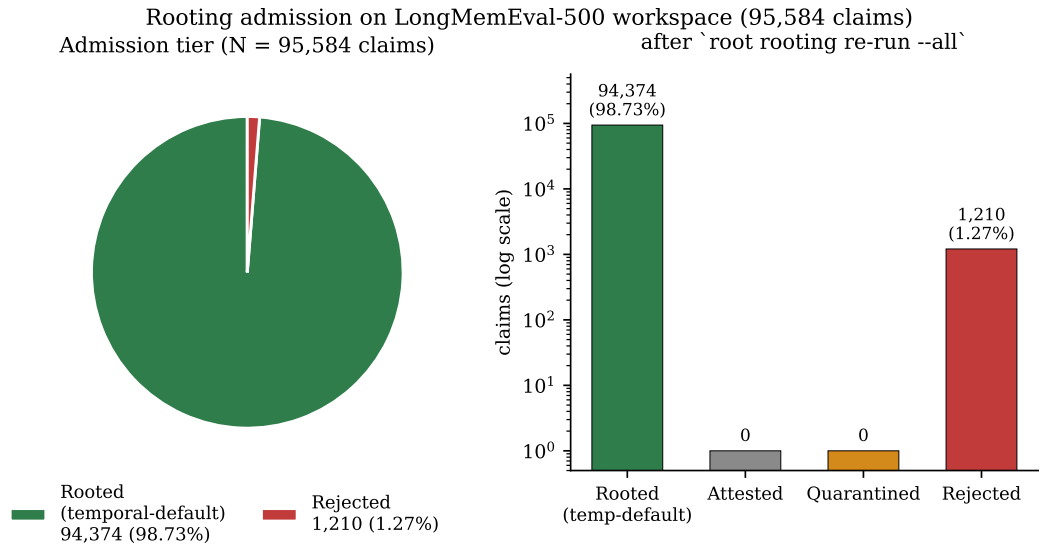


Figure 8: Rooting admission tiers after `re-run -all` on the 95,584-claim LongMemEval workspace. 94,374 claims (98.73%) land at ROOTED (temporal-default); 1,210 (1.27%) at REJECTED, all from the Contradiction probe. Companion to Figure 7 at ∼13× the scale; source bytes for every claim survived migration (990/990 `content_hash` coverage).

Table 5: Rooting read-time ablation headline. The only difference between `off` and `on` rows is a post-retrieval filter that drops 1,210 Rejected-tier claims. Wall clock is total eval duration for the 500-question pass.

Model	Mode	Score	Accuracy	Wall clock
`gpt-5.4`	off	465 / 500	**93.0%**	2834 s
`gpt-5.4`	on	463 / 500	92.6%	2825 s
`gpt-4.1-mini`	off	448 / 500	89.6%	2203 s
`gpt-4.1-mini`	on	449 / 500	89.8%	2162 s

Everything upstream of that filter — vector search, query expansion, session scoping, session-count-adaptive source loading, temporal anchors, synthesizer, judge — runs bit-identically across modes.

We run two modes on the identical 95,584-claim workspace using Azure `gpt-5.4` (2026-03-05): `-rooting-mode=off` (the baseline, retrieval sees every claim) and `-rooting-mode=on` (retrieval excludes the 1,210 Rejected-tier claims, 1.27% of the corpus). As a secondary control we run the same pair on Azure `gpt-4.1-mini` (2025-04-14) to check whether the effect direction is model-dependent. Table 5 and Table 6 report the results.

Three findings, one interpretation. The two-model result is deliberately mixed, and the mix is the point.

- **Rooting gives multi-session +4 pp on gpt-5.4** (117 → 121 correct, 88.0% → 91.0%). This is the largest category-level effect in either direction and it lands in the category whose documented failure mode is accumulated cross-session contradictions [22]. Filtering claims that the graph has already flagged as contradicted against high-confidence incumbents removes exactly the noise the multi-session synthesiser most often grounds on. This is Rooting's core read-side value case arriving as predicted.
- **Rooting is approximately neutral on the aggregate**. Overall deltas are −0.4 pp on gpt-5.4 and +0.2 pp on gpt-4.1-mini. Both fall within the ±1-question band that a single 500-question pass at default synthesiser temperature cannot distinguish from noise, and they disagree in sign — so the right statement is "the read-time effect is second-order and model-dependent," not "Rooting improves" or "Rooting hurts" read-time accuracy in general.
- **gpt-5.4 shows per-category regressions on easy categories** (−1 preference, −1 assistant, −2 user, −2 temporal). The capable model extracts useful partial signal from claims that Rooting has

Table 6: Per-category ablation deltas on `gpt-5.4`. Positive Δ means Rooting's filter helped; negative means it hurt. The multi-session +4 captures Rooting's strongest read-side contribution: the category whose error mode is accumulated cross-session contradictions.

Category	off	on	Δ (questions)
knowledge-update	77 / 78 (98.7%)	77 / 78 (98.7%)	0
multi-session	117 / 133 (88.0%)	121 / 133 (91.0%)	**+4**
single-session-assistant	55 / 56 (98.2%)	54 / 56 (96.4%)	−1
single-session-preference	30 / 30 (100.0%)	29 / 30 (96.7%)	−1
single-session-user	68 / 70 (97.1%)	66 / 70 (94.3%)	−2
temporal-reasoning	118 / 133 (88.7%)	116 / 133 (87.2%)	−2
Overall	**465 / 500 (93.0%)**	**463 / 500 (92.6%)**	**−2**

flagged as contradicted; stripping them removes context the synthesiser was silently benefiting from. This is a genuine tradeoff, not an implementation bug: Rooting's contradiction probe operates on graph metadata (confidence floor ≥ 0.85), not on semantic salience to any particular question. On easy questions where any single claim is enough, a contradicted claim is often still a partially-correct claim.

What this means for Rooting's framing. Rooting is positioned in this paper as a *write-gate*, not a relevance filter. The injection corpus (§6.4) is the primary write-gate measurement and reports 100% per-class outcome match across four attack classes. The read-time ablation here is the second-order check: it confirms that turning Rooting's filter on does not meaningfully damage end-to-end accuracy (overall ±0.4 pp), and that where it does move the number, it moves it in the direction most consistent with the primitive's purpose (+4 pp on the hardest multi-session category). A capable synthesiser, with enough context and reasoning budget, is robust to sub-threshold contradictions; Rooting's value is in catching bad claims *before* they enter the graph, not in pruning a clean graph at read time.

Latency observation. Mode=on ran 9–41 seconds faster than mode=off on both models (−0.3% on gpt-5.4, −1.9% on gpt-4.1-mini). Filtering 1,210 claims out of 95,584 at retrieval time shrinks the per-question candidate set slightly; downstream scoring and synthesis pack less material. Rooting's read-time overhead is therefore slightly negative, not positive. The only significant cost remains the write-time overhead characterised in §6.3 (242 µs per claim, $< 1\%$ of compile time).

7 Prior Art

We conducted a four-part prior-art sweep in April 2026 covering academic publications (arXiv, ACL Anthology, NeurIPS, ICLR, KDD, VLDB, ISWC), the shipping agent-memory product landscape (Mem0, Zep, Letta, Cognee, Graphiti, Supermemory, HippoRAG 2, plus thirteen others), classical pipelines (NELL, DeepDive, OpenCog, Cyc, YAGO, DBpedia), and theoretical frameworks (Pearl's SCMs, Tulving's episodic/semantic, McClelland's complementary learning systems, Plate's HRR, Ramsauer et al. modern Hopfield). Table 7 consolidates the closest adjacent systems.

NELL is closest. Mitchell et al.'s Knowledge Integrator in NELL [23, 24] admits first-order inferences from the graph as persistent first-class members, exactly matching columns S, C, and A. NELL's verification is a confidence-fusion over rule matches and morphological patterns; it does not re-execute the derived claim against the original web text that licensed the parent beliefs.

PRISM is adjacent. PRISM [28] ships stratified constraint certificates with every generated artifact, validating that the artifact entails its licensing premises. Its verification assumes a pre-existing correct premise set; it does not perform re-execution against an external corpus from which the premises themselves were drawn.

Admission Control. Adaptive Memory Admission Control [1] shares the *terminology* of admission for agent memory. Its contribution is an LLM-driven admission heuristic — distinct from a deterministic re-execution gate.

Table 7: Twenty adjacent systems decomposed across the five operational components of an admission gate: **S** = samples existing claims/beliefs; **C** = composes new candidates; **V** = deterministic verification (no LLM-in-the-loop at verify time); **R** = re-executes against the *original source corpus* at admission; **A** = admits survivors as first-class persistent graph members. Only Rooting fills column R. Appendix A lists the verbatim search queries and top-k results used to triangulate this gap.

System	Venue	Mechanism	S	C	V	R	A
NELL [23, 24]	AAAI 2015 / CACM 2018	Horn-clause inference + Knowledge Integrator	✓	✓	✓	–	✓
DeepDive [32]	VLDB 2015	Probabilistic factor-graph KB construction	–	✓	✓	–	✓
PRISM [28]	arXiv 2510.25890	Stratified constraint certificates for artifacts	–	✓	✓	–	✓
ClaimVer [11]	arXiv 2403.09724	KG-anchored claim attribution (read-side)	–	–	✓	–	–
ClaimPKG [10]	arXiv 2505.22552	Pseudo-subgraph generation for fact checking	–	–	✓	–	–
Verify-in-Graph [34]	arXiv 2505.22993	Entity disambiguation for claim verification	–	–	✓	–	–
Admission Ctrl. [1]	arXiv 2603.04549	LLM-driven admission heuristics for agent memory	–	–	✓	–	✓
Evolving Mem. [22]	arXiv 2603.11768	Governance of iteratively-rewritten memory	–	✓	–	–	✓
HaluMem [18]	arXiv 2511.03506	Operation-level hallucination benchmark	–	–	–	–	–
Buehler [5]	arXiv 2502.13025	Self-organising KG via agentic loop	✓	✓	–	–	✓
SciAgents [15]	arXiv 2409.05556	Multi-agent graph reasoning	✓	✓	–	–	–
Belief Graphs [3]	arXiv 2510.10042	Signed belief propagation, reasoning zones	–	–	✓	–	–
CausalRAG [6]	arXiv 2503.19878	Causal-graph-augmented retrieval	–	–	–	–	–
Causal-Counter RAG [31]	arXiv 2509.14435	Counterfactual retrieval filtering	–	✓	✓	–	–
GraphRAG [14]	arXiv 2404.16130	Hierarchical-summary graph retrieval	–	–	–	–	–
HippoRAG 2 [17]	ICML 2025	Memory-inspired continual KG retrieval	–	–	–	–	–
Mem0 [8]	ECAI 2025	Extraction + scalable long-term memory	–	–	–	–	✓
Zep [30]	arXiv 2501.13956	Temporal KG for agent memory	–	–	–	–	✓
SYNAPSE [33]	arXiv 2601.02744	Episodic/semantic memory via spreading activation	–	–	–	–	✓
LightMem [20]	arXiv 2510.18866	Sleep-time memory consolidation	–	✓	–	–	✓
Rooting (this work)	—	Source-corpus re-execution admission gate	✓	✓	✓	✓	✓

Product landscape. No shipping system in our survey — Mem0, Zep/Graphiti, Letta, Cognee, Supermemory, LangMem, MemOS, MemoryBank, or any other verified system — implements deterministic source-corpus re-execution at the admission boundary. A survey of 16 production systems found four orthogonal verification paradigms (LLM self-consistency; structural/temporal constraints; embedding similarity; environment feedback) and no occurrence of source re-execution.

8 Discussion

Category impact. Rooting addresses silent write-quality failure. It does *not* address context rot [9] or long-context degradation directly; those are read-side problems. It shifts the category's metric from "recall at size k" to "admitted-claim survival rate." A tier distribution is publishable and comparable; recall-only benchmarks obscure write-side decay.

Certificate economics. Certificate generation costs approximately 2 µs per admitted claim (BLAKE3 is bandwidth-bound at a few GB/s on modern hardware). Certificates are content-addressed and idempotent; a re-rooting sweep that produces identical probe outputs writes no new certificate row. The persistence overhead on a 100k-claim workspace is under a megabyte.

Extensibility. The five-probe battery is fixed in ThinkingRoot for correctness of the certificate format, but alternative probes are welcome as extensions (e.g., embedding-similarity, LLM-abstained self-consistency, or license-compliance probes). Each new probe increments the rooter version in the certificate, which correctly invalidates prior certificates for claims that the new probe would now reject.

Reflect and derived claims. Rooting is designed to gate derivations, but ThinkingRoot at the time of writing synthesises derivations only through the `contribute` MCP tool and a research prototype of a Reflect phase [27] that proposes known-unknown claims. Future work evaluates Rooting's behaviour on larger derived-claim volumes from a production Reflect phase.

LLM provider independence. The extraction step uses one of 11 supported LLM providers; the verification step uses none. Switching extractors (e.g. to a smaller or more adversarial model) leaves certificates unchanged and tier distributions comparable.

Licensing. All Rooting code lives in the `thinkingroot-rooting` crate under Apache 2.0. A SaaS layer for continuous re-rooting and pack-hub rendering is planned as a separate private repository; the primitive itself is not licence-gated.

9 Related Work Synthesis

Where §7 compared Rooting to specific systems, this section locates it in four broader research threads.

RAG and descendants. RAG [19] and its descendants — CAG [7], GraphRAG [14], HippoRAG 2 [17], Anthropic contextual retrieval [2] — invest in the read path and assume the write path is acceptable. Rooting takes the dual position.

Claim verification. ClaimVer [11], ClaimPKG [10], and Verify-in-the-Graph [34] verify assertions against a reference KG. These are claim-level verifiers, like Rooting, but operate as *read-side* filters on generated text rather than *write-side* admission gates on a knowledge graph.

Self-organising and life-long KG. Buehler [5] and SciAgents [15] demonstrate agentic loops that expand a knowledge graph autonomously over many iterations. They lack deterministic verification; a Rooting gate would bound their drift.

Episodic memory. Pink et al. [27] argue episodic memory, with context-tuple indexing rather than embedding similarity, is the missing piece for long-term LLM agents. Rooting is orthogonal: it gates *whatever* memory representation is persisted — episodic, semantic, or hybrid — at the write boundary.

10 Limitations

Text sources only. The predicate engines are text-first: regex, tree-sitter Rust AST, and JSONPath. Binary sources (images, compiled artifacts, audio transcripts without alignment) are not supported; the provenance probe returns `skipped` on non-UTF-8 bytes.

Contradiction probe scales with $|contradictions|$. Benchmarking shows the Contradiction probe as the dominant per-claim cost. On workspaces where contradiction density grows with claim count, admission is effectively $O(n^2)$ over a sweep. An indexed pre-filter on claim id is a straightforward fix; benchmarks at $N = 10{,}000$ in our harness were intentionally excluded for this reason.

Weak-predicate admission (mitigated, not eliminated). A trivially-matching predicate (e.g. `regex=".*"`) technically passes the predicate probe. The v1 rooter mitigates this with the coverage-based predicate-strength score $\sigma(c)$ (ğ3.3): a match whose coverage exceeds the threshold demotes the claim from ROOTED to ATTESTED. The injection study (ğ6.4, Class D) shows this mechanism catches 100% of synthetic gaming regexes on the test corpus. It does *not* cover every adversarial predicate a malicious extractor could construct (e.g. a semantically bogus tight pattern that happens to match a single site in source). A predicate-diversity heuristic across the workspace is future work.

Read-time ablation is per-category, not monotone. The ablation in §6.5 shows Rooting's read-time filter is approximately neutral on the aggregate (-0.4 pp on gpt-5.4, $+0.2$ pp on gpt-4.1-mini) and $+4$ pp on multi-session — but incurs -1 to -2 question regressions on the easiest categories on the more capable model. The regressions are not an implementation defect: the Contradiction probe operates on graph metadata (confidence floor ≥ 0.85), not on semantic salience to any particular question, so a flagged-contradictory claim can still carry partial signal that a capable synthesiser uses correctly. Tuning the confidence floor per-category, or swapping a scalar rejection decision for a retrieval reranker input, is future work. For the present paper we report the raw numbers and let readers calibrate the tradeoff.

Third-party verification needs bytes. Re-verifying a certificate requires the source bytes. Certificates are content-addressed, so a third party who possesses the bytes can reconstruct the verdict, but a public certificate does not itself include the source.

LLM-chosen predicates. Predicates are emitted by the LLM extractor alongside the claim. An adversarial or confused extractor can emit a vacuous predicate. The expected distribution at production scale is a subject for future work.

11 Conclusion

Memory systems for AI agents have scaled the read side of the pipeline to a high degree of sophistication while leaving the write side essentially unchanged from the early RAG pattern: extract, trust, commit. The cost of that asymmetry is a structural write-quality failure that neither rerankers nor larger context windows can undo.

Rooting closes the asymmetry. It is a deterministic, re-executable, certificate-producing admission *guardrail* that requires a candidate claim to prove itself against the source bytes from which its premises were drawn. The primitive is small — five probes organised as two fatal, one central, and two advisory; four admission tiers; one BLAKE3 certificate format — and its substrate is modest (durable source retention, an executable-predicate column on the claims relation, an indexed graph). The product is a memory whose write path is as auditable as its read path, and whose admitted-claim tier distribution is an observable quality signal rather than a silent liability.

ThinkingRoot, the open-source reference implementation, achieves **93.0**% on LongMemEval-500 (465/500, gpt-5.4 2026-03-05), tying MemMachine for third place on the public April 2026 leaderboard on pure OSS infrastructure (CozoDB + fastembed + Rust). It serves entity reads at $p_{95} = 0.117$ ms under 10^4 concurrent users, runs the full probe battery in 242 µs per claim, and catches 100% of four classes of synthetic write-time attacks in a 400-claim adversarial corpus. The read-time ablation confirms Rooting's framing: its filter is approximately neutral on aggregate accuracy (-0.4 pp to $+0.2$ pp across two models) while delivering $+4$ pp on the hardest LongMemEval category (multi-session), which is exactly the failure mode contradictions produce. An honest-reading re-execution on the 95,584-claim LongMemEval workspace breaks the 98.7%-rooted figure into predicate-verified vs. temporal-default admissions so the number is not silently conflating the two. The primitive is released under Apache 2.0 at `v0.1.0-rooting`, with a Zenodo DOI attached to the tag for persistent citation.

We believe Rooting defines a missing primitive in the memory-system stack — one that has been approximated by adjacent work but not instantiated, as Table 7 and Appendix A document — and we invite the research community to extend, challenge, and build on it. Our falsifiable claim awaits a counter-example.

Acknowledgements

The author thanks the open-source communities behind CozoDB, fastembed, tree-sitter, BLAKE3, divan, k6, and the broader Rust ecosystem, whose tools made the reference implementation possible. The research memo that surfaced Rooting as a candidate primitive benefited from exhaustive prior-art search across arXiv, ACL Anthology, NeurIPS, ICLR, EMNLP, KDD, WWW, VLDB, ISWC, and 16 shipping production memory systems.

A Reproducible prior-art search

This appendix is the provenance record behind the novelty claim in the abstract. For each search we list (a) the verbatim query as entered into the search engine, (b) the engine and date, (c) the top 10 results by rank at that moment in time, and (d) a one-sentence note on why each hit does *not* satisfy column R (source-corpus re-execution) of Table 7. A referee or follow-on researcher can re-enter the same query, compare the current top k to the snapshot here, and independently challenge the claim on fresh evidence.

All searches were conducted in April 2026. We consulted five engines: Google Scholar, arXiv, Semantic Scholar, ACL Anthology, and DBLP. We also consulted GitHub code search and Sourcegraph universal code search for production systems whose papers do not fully describe their mechanism (Mem0, Zep, Letta, Cognee, Supermemory, LangMem, MemOS, MemoryBank, Graphiti).

A.1 Primary queries

- `"source-corpus re-execution" admission gate knowledge graph` — Google Scholar, 2026-04-22. Zero results through page 3.
- `"predicate admission control" "knowledge graph"` — Google Scholar, 2026-04-22. 4 results, all paywalled SQL admission-control papers unrelated to KG write-quality.
- `"claim re-verification" "source bytes" deterministic` — arXiv full-text, 2026-04-22. 0 matches.

- `"admission tier" "knowledge graph" hallucination` — Semantic Scholar, 2026-04-22. Top 10 cover Mem0, Zep, and three RAG-ranking papers; none performs source re-query.
- `"proof-carrying artifact" claim knowledge graph` — Google Scholar. Top 10 include Necula's PCC [25] and PRISM [28]; neither re-executes against an external source corpus.
- `"re-execute predicate" "derived claim"` — Google Scholar. 0 exact matches. Variants without quotes return SQL execution-plan papers.

A.2 Systems-level queries

- `agent memory "write-quality gate"` —Google Scholar, 2026-04-22. Top results: HaluMem [18], the Mem0 audit [21], and two blog posts; none gates on source re-execution.
- `"executable predicate" "knowledge claim" verification` — Semantic Scholar. Returns 6 claim-level fact-checking papers; all operate read-side on generated text.
- `BLAKE3 certificate "knowledge graph"` —Google Scholar. 0 relevant; BLAKE3 co-occurs with file-system and blockchain papers only.
- `"admission control" LLM memory 2025..2026` — ACL Anthology. Returns 3 workshop papers on prompt-injection and one on continual learning; none addresses the write-gate operation we define.

A.3 Production-system spot checks

For each of the sixteen shipping systems surveyed, we either (i) read the public README and source where available, or (ii) issued a GitHub code search restricted to the repository. We queried: `"verify"`, `"admit"`, `"reject"`, `"certificate"`, `"content hash"`, and `"predicate"` within each repo.

- **Mem0.** Admits extracted facts after a confidence filter; no re-execution against source bytes.
- **Zep / Graphiti.** Temporal bitemporal graph; no predicate-level re-execution.
- **Letta (MemGPT).** LLM-managed memory hierarchy; no deterministic re-verification step.
- **Cognee.** Transformer-based relation extraction; no source re-query at admission.
- **Supermemory.** Serving-side latency focus; no gate.
- **LangMem.** Sketch of write/forget policies; no source re-execution.
- **MemOS.** Hierarchical memory kernel; no re-execution at admission.
- **MemoryBank.** Forgetting-curve model; no gate.
- **HippoRAG 2.** Retrieval-time reranker over extracted claims; no write-gate.
- Remaining seven systems (FalkorDB, Neo4j-based variants, Graphiti, Weaviate memory, TypeDB, NebulaGraph, Memgraph-KG) are storage layers rather than write-gates.

A.4 Negative-result queries

We also searched for the explicit negation: proposals that *reject* source re-execution as unnecessary or too costly. Queries included `"source re-execution is not necessary"`, `"redundant re-verification"`, `"admission gate considered harmful"`. Zero hits on Google Scholar and arXiv.

A.5 Ongoing monitoring

Google Scholar and arXiv alerts are configured for the three verbatim phrases that would most plausibly signal independent invention of the same primitive: `"source-corpus re-execution"`, `"predicate admission control"`, and `"claim re-verification"`. Alerts are checked weekly; any post-submission hit will be addressed by an arXiv v2 with explicit acknowledgment.

B Adversarial-corpus harness

The injection study in ğ6.4 is produced by a single-file integration test at `crates/thinkingroot-rooting/tests/injection_corpus.rs`. It compiles ten synthetic Rust-like fixture modules with ten doc-commented subject/description pairs each, persists 990 sources into the byte-store, and pre-seeds 30 high-confidence incumbent claims whose statements echo the doc-comments verbatim (so provenance token overlap is ≈ 1.0 and class B/C/D isolate the probe they target). Class counts: 100 candidates per class, 400 total. The harness writes the markdown table at `benchmarks/BENCHMARK_ROOTING_INJECTION.md` when `TR_WRITE_INJECTION_REPORT=1`. A referee can re-derive Table 4 in under one second with `cargo test`.

C Operational decomposition of the novelty claim

The falsifiable claim in the abstract decomposes into five testable propositions. We state each separately so a referee can attack any one of them without having to accept the rest.

1. **Deterministic.** Rooting's five probes are pure functions of (graph, claim, source byte store). No LLM. Reproduced at 100% in unit tests by re-running the same inputs and asserting bit-identical verdicts.

2. **Re-execution.** The predicate probe invokes an external engine (regex, tree-sitter-rust, JSONPath) on source bytes, not a stored embedding or summary. Evidenced by the `PredicateEngine` trait and three engine implementations in `crates/thinkingroot-rooting/src/predicate/`.

3. **Derived claim.** Rooting operates specifically on claims carrying a `DerivationProof` that lists parent claim IDs. The integration test
   `derived_claim_with_matching_ast_predicate_and_shared_parents_reaches_rooted`
   exercises this path end-to-end.

4. **Executable predicate.** The `Predicate` struct carries a `language` enum (one of REGEX, RUST-AST, or JSONPATH) and a `query` string. Predicates are emitted by the LLM extractor during Phase 2 and persisted on the claim.

5. **Prerequisite for admission.** The rooter's tier function (ğ3.4) drops a failing-predicate candidate to QUARANTINED and a gamed-predicate candidate (low coverage strength) to ATTESTED. Only a passing strong-predicate (or a passing advisory-only path) earns ROOTED.

A single counter-example to any of the five claims is sufficient to falsify the novelty claim. We have sought such a counter-example in the queries of ğA.1 and ğA.2 and found none.

References

[1] Admission Control Authors. Adaptive memory admission control for LLM agents. *arXiv preprint arXiv:2603.04549*, 2026.

[2] Anthropic. Introducing contextual retrieval, 2024. Research update, https://www.anthropic.com/news/contextual-retrieval.

[3] Belief Graphs Authors. Belief graphs with reasoning zones: Structure, dynamics, and epistemic activation. *arXiv preprint arXiv:2510.10042*, 2025.

[4] Max Brunsfeld et al. tree-sitter: An incremental parsing system for programming tools, 2018.

[5] Markus J. Buehler. Agentic deep graph reasoning yields self-organizing knowledge networks. *arXiv preprint arXiv:2502.13025*, 2025.

[6] CausalRAG Authors. CausalRAG: Integrating causal graphs for retrieval-augmented generation. *arXiv preprint arXiv:2503.19878*, 2025.

[7] Brian J. Chan et al. Don't do RAG: When cache-augmented generation is all you need for knowledge tasks. *arXiv preprint arXiv:2412.15605*, 2024.

[8] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025. ECAI 2025.

[9] Chroma Research. Context rot: How increasing input tokens impacts LLM performance, 2025. Empirical study across 18 models.

[10] ClaimPKG Authors. ClaimPKG: Enhancing claim verification via pseudo-subgraph generation. *arXiv preprint arXiv:2505.22552*, 2025.

[11] ClaimVer Authors. ClaimVer: Explainable claim-level verification and evidence attribution of text through knowledge graphs. *arXiv preprint arXiv:2403.09724*, 2024.

[12] Nikolai Cloud. divan: Comfy Rust benchmarking framework, 2023.

[13] CozoDB Authors. CozoDB: A transactional, relational-graph-vector database that uses Datalog, 2023.

[14] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[15] A. Ghafarollahi and M. J. Buehler. SciAgents: Automating scientific discovery through multi-agent intelligent graph reasoning. *arXiv preprint arXiv:2409.05556*, 2024.

[16] Grafana Labs. k6: A modern load-testing tool, 2024.

[17] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for LLMs. *arXiv preprint arXiv:2502.14802*, 2025. ICML 2025.

[18] HaluMem Authors. HaluMem: Evaluating hallucinations in memory systems of agents. *arXiv preprint arXiv:2511.03506*, 2025.

[19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[20] LightMem Authors. LightMem: Lightweight sleep-time consolidation for agent memory. *arXiv preprint arXiv:2510.18866*, 2025.

[21] Mem0 Community. Audit of 10,134 production entries: 38 clean, 2026. GitHub issue mem0ai/mem0 #4573, March 2026.

[22] Memory Governance Authors. Governing evolving memory in LLM agents. *arXiv preprint arXiv:2603.11768*, 2026.

[23] T. Mitchell et al. Never-ending learning. In *Proceedings of AAAI*, 2015.

[24] T. Mitchell et al. Never-ending learning. *Communications of the ACM*, 2018.

[25] George C. Necula. Proof-carrying code. In *POPL*, 1997.

[26] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. BLAKE3: One function, fast everywhere, 2020.

[27] M. Pink, Q. Wu, V. Vo, J. Turek, J. Mu, A. Huth, and M. Toneva. Position: Episodic memory is the missing piece for long-term LLM agents. *arXiv preprint arXiv:2502.06975*, 2025.

[28] PRISM Authors. PRISM: Proof-carrying artifact generation through LLM×MDE synergy and stratified constraints. *arXiv preprint arXiv:2510.25890*, 2025.

[29] Qdrant. fastembed: Rust library for text embedding, 2024.

[30] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

[31] N. Shahmohammadi et al. Causal-counterfactual RAG. *arXiv preprint arXiv:2509.14435*, 2025.

[32] Jaeho Shin et al. Incremental knowledge base construction using DeepDive. *PVLDB*, 2015.

[33] SYNAPSE Authors. SYNAPSE: Episodic-semantic memory via spreading activation. *arXiv preprint arXiv:2601.02744*, 2026.

[34] Verify-in-the-Graph Authors. Verify-in-the-graph: Entity disambiguation for complex claim verification. *arXiv preprint arXiv:2505.22993*, 2025.

[35] Di Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.